

Amrit Chhaya Foundation - Frontend Integration & API Guide

This guide is designed for the frontend developer to easily integrate and interact with the **Amrit Chhaya Foundation** backend API.

☐ 1. Environment & Base URL Setup

Set up your frontend environment variable (`.env` or `.env.local` for Vite / Next.js / React):

```
VITE_API_BASE_URL=http://localhost:5000/api/v1
```

For production deployment:

```
VITE_API_BASE_URL=https://api.amritchhayafoundation.org/api/v1
```

☐ 2. API Response & Error Conventions

All API responses follow a strict, predictable JSON format.

☐ Successful Response Format (HTTP 200 / 201)

```
{
  "success": true,
  "message": "Operation successful",
  "data": {}
}
```

☐ Error Response Format (HTTP 400 / 401 / 403 / 404 / 500)

```
{
  "success": false,
  "message": "Human readable error message",
  "error": {
    "code": "ERROR_CODE_STRING",
    "details": {}
  }
}
```

□ Standard Error Codes

Code	Meaning	HTTP Status
AUTH_REQUIRED	Missing or invalid authentication token	401
INVALID_TOKEN	Expired or corrupted access token	401
INVALID_CREDENTIALS	Incorrect email or password	401
PERMISSION_DENIED	User lacks admin privileges	403
RESOURCE_NOT_FOUND	Resource (Pooja, Yatra, etc.) not found	404
BOOKING_NOT_FOUND	Booking ID does not exist	404
VALIDATION_ERROR	Missing required payload fields	400
PAYMENT_VERIFICATION_FAILED	Invalid Razorpay cryptographic signature	400

□ 3. Recommended Frontend Axios API Client Setup

To avoid repeating headers and token management, create an `apiClient.js` in your React project:

```

// src/api/apiClient.js
import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://localhost:5000/api/v1';

const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request Interceptor: Automatically attach JWT Access Token if available
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('accessToken');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
}, (error) => Promise.reject(error));

// Response Interceptor: Handle automatic token refreshing on 401
apiClient.interceptors.response.use(
  (response) => response.data,
  async (error) => {
    const originalRequest = error.config;
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      try {
        const refreshToken = localStorage.getItem('refreshToken');
        if (refreshToken) {
          const res = await axios.post(`${API_BASE_URL}/auth/refresh`, { refreshToken });
          if (res.data.success) {
            localStorage.setItem('accessToken', res.data.data.accessToken);
            originalRequest.headers.Authorization = `Bearer ${res.data.data.accessToken}`;
            return apiClient(originalRequest);
          }
        }
      } catch (refreshErr) {
        localStorage.removeItem('accessToken');
        localStorage.removeItem('refreshToken');
        window.location.href = '/admin/login';
      }
    }
  }
);

```

```
    return Promise.reject(error.response?.data || error.message);
  }
});

export default apiClient;
```

□ 4. Authentication APIs

4.1 Admin Login

POST /auth/login

Request Body:

```
{
  "email": "admin@example.com",
  "password": "password"
}
```

React Example:

```
import apiClient from '../api/apiClient';

async function handleAdminLogin(email, password) {
  try {
    const response = await apiClient.post('/auth/login', { email, password });
    if (response.success) {
      localStorage.setItem('accessToken', response.data.accessToken);
      localStorage.setItem('refreshToken', response.data.refreshToken);
      console.log('Logged in as:', response.data.user);
    }
  } catch (error) {
    alert(error.message || 'Login failed');
  }
}
```

□ 5. Pooja & Slot APIs

5.1 Get All Poojas (Paginated)

```
GET /poojas?page=1&limit=10&featured=true&category=dosha-puja&search=kaal
```

Response:

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "name": "Kaal Sarp Dosh Puja",
        "slug": "kaal-sarp-dosh-puja",
        "shortDescription": "Special puja for Kaal Sarp Dosh",
        "image": "https://images.unsplash.com/...",
        "startingPrice": 5100,
        "duration": "2 Hours",
        "featured": true
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 10,
      "total": 25,
      "totalPages": 3
    }
  }
}
```

5.2 Get Single Pooja Details

```
GET /poojas/:slug
```

Example: GET /poojas/kaal-sarp-dosh-puja

Response:

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Kaal Sarp Dosh Puja",
    "slug": "kaal-sarp-dosh-puja",
    "description": "Complete description...",
    "image": "https://images.unsplash.com/...",
    "duration": "2 Hours",
    "category": { "id": 1, "name": "Dosha Puja" },
    "variants": [
      { "id": 1, "name": "Basic", "price": 5100 },
      { "id": 2, "name": "With Prasad", "price": 7100 }
    ],
    "features": ["Pandit Ji", "Puja Samagri", "Prasad"]
  }
}
```

5.3 Fetch Available Slots for a Date

```
GET /poojas/:poojaId/slots?date=2026-09-20
```

Response:

```
{
  "success": true,
  "data": {
    "date": "2026-09-20",
    "slots": [
      { "id": 1, "startTime": "09:00", "endTime": "11:00", "available": true },
      { "id": 2, "startTime": "12:00", "endTime": "14:00", "available": false }
    ]
  }
}
```

☐ 6. Complete Booking & Razorpay Payment Integration

[!IMPORTANT] PRICE SECURITY RULE (Section 47): The frontend **MUST NOT** calculate or submit *amount* in the request body. Send only *poojaId* and *variantId*. The backend retrieves the authoritative price from the database!

Step 1: Create Booking

POST /bookings

Request Body:

```
{
  "poojaId": 1,
  "variantId": 2,
  "slotId": 1,
  "date": "2026-09-20",
  "devotee": {
    "name": "Jatin Jain",
    "phone": "9999999999",
    "email": "jatin@example.com",
    "gotra": "Kashyap",
    "address": {
      "line1": "Mahakal Marg",
      "city": "Ujjain",
      "state": "Madhya Pradesh",
      "pincode": "456001",
      "country": "India"
    }
  },
  "specialRequest": "Please perform the puja in the morning.",
  "prasadRequired": true
}
```

Response:

```
{
  "success": true,
  "message": "Booking created successfully",
  "data": {
    "bookingId": "ACF-2026-792058",
    "bookingStatus": "PENDING_PAYMENT",
    "amount": 7100,
    "currency": "INR",
    "paymentRequired": true
  }
}
```

Step 2: Create Razorpay Order

POST /payments/create-order

Request Body:

```
{
  "bookingId": "ACF-2026-792058"
}
```

Response:

```
{
  "success": true,
  "data": {
    "orderId": "order_h5sb7m6731",
    "amount": 710000,
    "currency": "INR",
    "keyId": "rzp_test_sample_key"
  }
}
```

Step 3 & 4: Open Razorpay Checkout & Verify Signature

Below is the complete, copy-pasteable React payment flow handler:

```

// React Booking Payment Component Example
import apiClient from '../api/apiClient';

async function initiatePoojaPayment(bookingPayload) {
  try {
    // 1. Create Booking on Backend
    const bookingRes = await apiClient.post('/bookings', bookingPayload);
    const { bookingId } = bookingRes.data;

    // 2. Create Razorpay Order on Backend
    const orderRes = await apiClient.post('/payments/create-order', { bookingId });
    const { orderId, amount, keyId, currency } = orderRes.data;

    // 3. Configure Razorpay Options
    const options = {
      key: keyId,
      amount: amount, // Amount in paise
      currency: currency,
      name: "Amrit Chhaya Foundation",
      description: "Pooja Booking Payment",
      order_id: orderId,
      handler: async function (response) {
        // 4. Verify Payment on Backend
        try {
          const verifyRes = await apiClient.post('/payments/verify', {
            bookingId: bookingId,
            razorpayOrderId: response.razorpay_order_id,
            razorpayPaymentId: response.razorpay_payment_id,
            razorpaySignature: response.razorpay_signature
          });

          if (verifyRes.success) {
            alert('Payment Successful! Booking Confirmed: ' + bookingId);
            window.location.href = `/booking-success?id=${bookingId}`;
          }
        } catch (verifyError) {
          alert('Payment verification failed. Please contact support.');
```

```
    theme: {
      color: "#D97706"
    }
  };

  // Open Razorpay Popup
  const rzp = new window.Razorpay(options);
  rzp.open();

} catch (error) {
  console.error('Booking failed:', error);
  alert(error.message || 'Booking process failed');
}
}
```

□ 7. Other Public APIs Overview

□ 7.1 Yatras

- **Get All Yatras:** GET /yatras
- **Get Yatra Details:** GET /yatras/:slug
- **Book Yatra:**

```
POST /yatra-bookings
```

```
{
  "yatraId": 1,
  "dateId": 5,
  "numberOfPeople": 2,
  "customer": {
    "name": "Jatin Jain",
    "phone": "9999999999",
    "email": "jatin@example.com"
  }
}
```

□ 7.2 Donations

- **Get Donation Categories:** GET /donation-categories
- **Create Donation Order:**

```
POST /donations
```

```
{
  "categoryId": 1,
  "amount": 1100,
  "donor": {
    "name": "Jatin Jain",
    "phone": "9999999999",
    "email": "jatin@example.com"
  },
  "anonymous": false
}
```

- **Get Receipt URL:** GET /donations/:donationId/receipt

□ 7.3 Goshala (Cows)

- **Get Cows List:** GET /goshala/cows

□ 7.4 Content APIs

- **Gallery:** GET /gallery?category=Pooja&page=1&limit=10
- **Blogs List:** GET /blogs?page=1&limit=10
- **Blog Single:** GET /blogs/:slug
- **Events List:** GET /events
- **Event Single:** GET /events/:slug
- **Testimonials:** GET /testimonials
- **Submit Enquiry:**

```
POST /enquiries
```

```
{
  "name": "Jatin Jain",
  "phone": "9999999999",
  "email": "jatin@example.com",
  "message": "I want information about Ujjain Darshan."
}
```

□ 8. Protected Admin APIs

Note: Include Authorization: Bearer <token> in headers (handled automatically if using apiClient.js above).

- **Admin Dashboard Stats:** GET /admin/dashboard
- **Get All Bookings:** GET /admin/bookings?status=CONFIRMED&page=1&limit=20
- **Update Booking Status:** PATCH /admin/bookings/:id/status { "status": "COMPLETED" }
- **Get Customers:** GET /admin/customers
- **Customer Profile:** GET /admin/customers/:id
- **Create Pooja:** POST /admin/poojas
- **Update Pooja:** PUT /admin/poojas/:id
- **Delete Pooja:** DELETE /admin/poojas/:id
- **Get Enquiries:** GET /admin/enquiries
- **Update Enquiry:** PATCH /admin/enquiries/:id { "status": "CONTACTED", "notes": "Done" }

☐ Quick Checklist for Frontend Developer

- ☐ Include Razorpay script in index.html: `<script src="https://checkout.razorpay.com/v1/checkout.js"></script>`
- ☐ Set VITE_API_BASE_URL in .env
- ☐ Implement apiClient.js with auto bearer token & 401 refresh handling
- ☐ Always handle loading state, error toasts, and empty states
- ☐ Never pass hardcoded amount values when creating bookings!

